

NORTE MARKET

Documentação Oficial — AV1 Programação III — Full Stack

Semestre 2026/2 · Projeto integrador · Etapa 100% front-end

Tema: catálogo inteligente de produtos alimentado por API pública.

Integrantes: preencher com os nomes oficiais do grupo antes da entrega.

1. Visão geral

O Norte Market foi idealizado para aplicar, em uma solução única, os conteúdos das Aulas 02 a 05. A aplicação principal utiliza React + Vite, React Router, Axios e SCSS. Uma segunda aplicação contém a página institucional em Next.js + Tailwind CSS. O back-end próprio não faz parte desta etapa.

2. Arquitetura

apps/react — aplicação principal: componentes, páginas, rotas, consumo de API, SCSS e testes.

apps/institucional — página institucional em Next.js + Tailwind.

docs — documentação, roteiro e plano de evidências.

3. Aula 02 — Git, SCSS, Box Model e Flexbox

O projeto contém variáveis em `_variables.scss`, nesting em `main.scss` e os mixins `focus-ring` e `container`. O layout utiliza Flexbox no cabeçalho, navegação, ações e rodapé. O Box Model é aplicado com padding, margin, border e box-sizing: border-box.

4. Aula 03 — Responsividade e JavaScript

A aplicação usa meta viewport e abordagem mobile-first. Existem breakpoints em 640px e 960px, cobrindo celular, tablet e desktop. A lógica utiliza let/const, arrays, objetos, funções, condicionais, loops com map, filter e eventos como onChange, onClick e onSubmit.

5. Aula 04 — React, Hooks e Rotas

O projeto React é criado para Vite. Há componentes reutilizáveis como Header, Footer e ProductCard. O React Router apresenta as rotas `/`, `/catalogo`, `/favoritos` e `404`. `useState` controla filtros, favoritos, carregamento e o contador. As listas são renderizadas com `map()` e `key`.

6. Aula 05 — API, Next.js, Tailwind, testes e deploy

A tela de catálogo faz GET em `https://fakestoreapi.com/products` por Axios. `useEffect` dispara o carregamento e `useState` armazena os dados. A página institucional foi criada em Next.js e usa Tailwind CSS com breakpoints sm/md/lg. Dois testes Jest cobrem filtro e formatação de preço.

7. API pública e exemplo de dado

```
{ id, title, price, description, category, image }
```

8. Execução local

```
cd apps/react
npm install
npm run dev

npm test
```

Para a aplicação institucional: `cd apps/institutional && npm install && npm run dev`.

9. Deploy

Publicar as pastas `apps/react` e `apps/institutional` como projetos separados na Vercel, mantendo duas URLs públicas. Antes da entrega, inserir as URLs reais nesta documentação e no Classroom.

10. Evidências exigidas

O documento final deve incluir prints reais do GitHub, estrutura de pastas, SCSS, responsividade no DevTools, rotas, consumo da API, página Next/Tailwind, `npm test` com os testes passando e as duas URLs da Vercel.

11. Checklist final

Item	Status
Repositório público + histórico	■
SCSS: variáveis + nesting + mixin	■
Responsividade em 3 tamanhos	■
React: componentes + hooks + rotas	■
Axios GET + <code>useEffect</code> + <code>map</code>	■
Next.js + Tailwind sm/md/lg	■
2 testes Jest passando	■
2 URLs públicas na Vercel	■
Documentação + prints reais	■